

Relatório — Criando um Jogo com Agentes de Codificação

Jogo: Defesa do Castelo (Tower Defense Medieval) **Integrantes:** Josué Oliveira e Roberto Rawlison **Plataforma:** HTML5 Canvas + JavaScript puro (módulos ES6, sem frameworks) **Repositório:** github.com/robertorawlison/tower-defense **Data:** junho de 2026

1. Introdução

Este relatório descreve o desenvolvimento de um jogo feito com o apoio de um agente de codificação. A atividade sugeria o Tetris, mas optamos (com combinação prévia) por criar um jogo de **Tower Defense**: o jogador defende um castelo de ondas de bárbaros usando apenas torres automáticas. Ao derrotar inimigos, ganha moedas; com as moedas, compra e melhora torres. São 10 ondas, e sobreviver à última é a vitória.

Embora o jogo seja diferente do sugerido, ele exercita os mesmos conceitos pedidos em cada etapa do guia — campo de jogo, controle dos elementos, detecção de colisão, condição de fim de jogo, níveis e pontuação, tela de abertura, *attract mode*, *high scores*, persistência, estilos visuais e áudio. A Seção 3 mostra como cada etapa foi atendida.

O relatório está organizado em três partes principais: a experiência prévia do grupo (Seção 2), o relato do desenvolvimento (Seção 3) e as conclusões e comentários (Seção 4).

2. Experiência prévia

2.1. Linguagem e plataforma

A experiência do grupo com HTML e JavaScript era **básica**. Já tínhamos tido algum contato com a linguagem e entendíamos conceitos fundamentais (variáveis, funções, noções de página web), mas nenhum dos integrantes havia desenvolvido projetos próprios desse porte. Vários recursos usados no jogo eram novos para nós, como módulos ES6, renderização em `<canvas>`, o laço de animação com `requestAnimationFrame`, a Web Audio API e o armazenamento local (`localStorage`).

2.2. Desenvolvimento de jogos

Aqui a experiência era **nenhuma**: foi a primeira vez que o grupo construiu um jogo. Não tínhamos familiaridade com ideias comuns da área, como o *game loop* (o ciclo que atualiza e redesenha a tela a cada quadro), o *delta time* (medir o tempo entre quadros para que o

movimento não dependa da taxa de quadros), máquina de estados de telas, detecção de colisão, geração procedural de mapas e busca de caminhos (BFS).

Esse ponto de partida — alguma base em web, nada em jogos — ajuda a entender o restante do relatório: boa parte do que foi produzido só foi possível por causa do agente, e parte das nossas conclusões trata justamente de **quanto realmente entendemos** do que foi gerado.

3. Relato do desenvolvimento

3.1. Ferramentas e estrutura de agentes

- **Agente de codificação:** usamos o **Claude Code** (ferramenta de linha de comando da Anthropic) como um **agente único**, com acesso ao sistema de arquivos e ao terminal. Ele lia e escrevia os arquivos, rodava comandos (servidor local, Git) e conseguia verificar o próprio trabalho. Não montamos uma arquitetura com vários agentes.
- **Modelos:** modelos **Claude** da família 4.x — principalmente o **Sonnet** (mais rápido, usado na maioria das edições) e o **Opus** (em momentos de planejamento e tarefas mais elaboradas). Alternamos entre eles ao longo do projeto.
- **Imagens:** usamos o **ChatGPT** para gerar as imagens de referência dos sprites (personagens, torres, caminhos, castelo e decorações em pixel art). Depois recortamos esses sprites e os organizamos na pasta `assets/` que o jogo espera.
- **Ambiente:** Windows, com o jogo servido localmente por HTTP (necessário porque módulos ES6 não funcionam abrindo o arquivo direto pelo `file://`).

Um ponto marcante do fluxo foi o agente **fazer perguntas antes de programar**. Em vez de já sair gerando código, ele propôs um plano e levantou decisões de projeto (stack, estrutura do mapa, como o jogador perde, número de ondas, etc.), que respondemos. Essas respostas viraram um documento de plano que serviu de contexto para o resto do trabalho.

3.2. Etapa 1 — Base do jogo

Começamos com um prompt amplo descrevendo a ideia e fomos refinando com prompts complementares. Os principais foram:

"crie um código em javascript [...] um jogo de tower defense no qual você é um cavaleiro e tem que defender o castelo contra tropas invasoras de bárbaros, e seu objetivo é coletar moedas derrotando os bárbaros [...]"

"com essas moedas você pode comprar torres de vigia com arqueiros, torres com bestas [...] e depois uma catapulta que custa mais caro; são três tiers de armas"

"faça com que, ao passar das ondas, tenham outros tipos de bárbaros — quero que tenham 3 tipos"

"depois de cada onda o jogador terá uma 'pausa' para poder colocar as torres [...]"

"o mapa será gerado aleatoriamente, ele terá 'caminhos' que os bárbaros vão passar [...] as armas terão pontos fixos para serem colocadas 'ao redor' dos caminhos"

"agora, para o plano, faça perguntas e dê sugestões de como pode ser feito"

A partir das respostas que demos às perguntas do agente, fixamos as seguintes decisões:

- **Stack:** HTML5 Canvas + JavaScript puro (sem framework).
- **Sem cavaleiro controlável** — apenas torres automáticas.
- **Mapa:** grid com múltiplos caminhos, gerado proceduralmente.
- **Visual:** sprites PNG externos, com *fallback* de retângulos coloridos caso algum sprite não carregue (isso nos permitiu testar a mecânica antes de ter os sprites prontos).
- **Derrota:** castelo com HP (100). **Vitória:** sobreviver às 10 ondas.
- **Upgrade in-place** das torres.
- **Grid 36×21 células de 48px.**
- **Maapeamento dos inimigos:** martelo = básico, furioso = veloz, machado = tanque.

O resultado foi a base jogável: geração do mapa com caminhos, encaixe automático dos tiles de caminho (*autotile*), castelo com barra de HP, as 3 torres (arqueiro, besta, catapulta) com upgrade, os 3 tipos de bárbaro, projéteis (incluindo o tiro em arco da catapulta), as 10 ondas, a fase de construção entre ondas e as condições de vitória e derrota. O código foi organizado em módulos com responsabilidades separadas (`main`, `config`, `map`, `maze`, `tower`, `enemy`, `projectile`, `wave`, `ui`, `render`), o que facilitou muito as mudanças seguintes.

Em termos do guia: o "campo de jogo" é o mapa; o controle das peças vira o posicionamento e upgrade de torres; a detecção de colisão aparece no acerto dos projéteis e na chegada dos inimigos ao castelo; e a condição de fim de jogo é o HP do castelo chegar a zero.

3.3. Etapa 2 — Dificuldade e pontuação

Esta etapa, que no guia trata de níveis e pontuação, virou no nosso jogo:

- **Dificuldade (Fácil / Difícil):** uma tela de seleção no início que muda a quantidade e a disposição dos caminhos pelos quais os bárbaros chegam. Também adicionamos um botão para **regenerar o labirinto**.
- **Pontuação:** decidimos pontuar pela **vida do castelo preservada ao final**. Proteger o castelo 100% (sem perder vida) dá a nota máxima; abaixo disso, a pontuação cai proporcionalmente, com "medalhas" por faixa (Perfeito, Ouro, Prata, Bronze, Sobreviveu) e

Derrota = 0%.

- **High scores:** criamos um **ranking das melhores tentativas**, ordenado por vitória, pontuação e onda alcançada, destacando a tentativa atual e sinalizando novos recordes.

3.4. Etapa 3 — Tela de abertura e loop de arcade

Seguimos de perto o guia, inclusive os itens opcionais:

- **Tela de abertura:** o jogo abre numa tela de título e espera o jogador interagir (qualquer tecla ou clique) para começar.
- **Game over e volta ao início:** ao fim da partida (vitória ou derrota), o jogo mostra o resultado com a pontuação e depois retorna à tela de abertura.
- **Attract mode:** após um tempo parado na abertura, o jogo entra em modo de demonstração, no qual um "bot" joga sozinho. A demonstração termina assim que o jogador interage.
- **Loop de arcade:** implementamos o ciclo clássico — abertura → (após um tempo) *attract mode* → (após mais um tempo) tela de *high scores* → de volta para a abertura.

3.5. Etapa 4 — Persistência, visual, áudio e extras

Implementamos várias das adições sugeridas, além de extras próprios:

- **Persistência das pontuações:** o ranking é gravado no `localStorage` do navegador, então as melhores pontuações continuam disponíveis ao reabrir o jogo.
- **Estilos visuais:** o jogo usa **sprites PNG** para grama, caminhos, decorações, castelo, torres e inimigos, com **renderização direcional** (4 direções) para torres e bárbaros, mantendo o *fallback* de formas coloridas.
- **Efeitos sonoros e música:** adicionamos **música de fundo** (uma melodia medieval simples sintetizada) e vários **efeitos em MP3**, disparados em eventos específicos: matar um bárbaro, início de onda, início de partida (fácil/difícil), conclusão de onda, construção/upgrade de torre, dano ao castelo e os jingles de vitória e derrota. Há botão de mudo (atalho **M**), e o áudio respeita a regra dos navegadores de só iniciar após a primeira interação do usuário.
- **Efeitos visuais e extras:** indicadores pulsantes que mostram, antes da onda, **de onde os bárbaros vão surgir; **barra de HP destacada sobre o castelo**; modo de velocidade (**1x a 4x**); **ajuste fino da geração dos caminhos (deixá-los menos sinuosos)**; e um servidor local sem cache** para o desenvolvimento (ver Seção 3.6).

3.6. Retrabalho e dificuldades

Como pede o guia, registramos que algumas partes exigiram **retrabalho** e vários prompts de correção. Os casos mais relevantes:

- **Cache do navegador:** o problema mais confuso do projeto. Depois de algumas mudanças, os

botões "paravam de funcionar". A causa não era o código, e sim o navegador servindo uma mistura de arquivos novos e antigos (gerando um erro do tipo "o módulo não exporta tal função"). Resolvemos diagnosticando com um detector de erros na tela e, por fim, trocando o servidor por um que envia cabeçalhos `*no-cache*`.

- **Tamanho do mapa:** pedimos para diminuir o mapa, mas isso fez alguns caminhos serem gerados para fora da área visível; tivemos que reverter a mudança.

- **Calibragem dos caminhos:** o ajuste das curvas foi iterativo — pedimos para reduzir o "zig-zag" várias vezes em sequência até chegar no resultado desejado.

- **Frestas entre os tiles:** surgiram linhas claras entre os quadrados do mapa (um artefato de renderização), que exigiram alguns ajustes no desenho dos tiles.

Em geral foram correções pontuais, e o agente conseguiu diagnosticar e resolver a maioria a partir da descrição do sintoma (e, quando preciso, de um print da tela).

4. Conclusões e comentários

4.1. Impacto da falta de experiência

A falta de experiência com jogos **não impediu** o projeto de avançar — pelo contrário, o agente produziu uma base funcional e foi incrementando recurso sobre recurso. A base em web ajudou a acompanhar a estrutura geral (arquivos, ideia de funções), mas os conceitos específicos de jogos chegaram prontos pelo agente, sem precisarmos estudá-los a fundo de antemão.

Por outro lado, a inexperiência apareceu na hora de **avaliar e direcionar**. Em vários momentos só percebíamos que algo estava errado pelo resultado visual ("o caminho está bugado", "as linhas verdes", "o botão não responde") e dependíamos do agente para entender a causa. Se fôssemos continuar o projeto de forma séria, seria necessário estudar mais — tanto a linguagem quanto fundamentos de jogos — para decidir com mais autonomia e revisar melhor o que é gerado.

4.2. O que aprendemos, e seria possível sem agentes?

Aprendemos bastante sobre **como um jogo é estruturado** na prática: o game loop rodando a cada quadro, a separação em módulos, o uso de `localStorage` para persistência, a Web Audio

API para som e o porquê de um jogo em módulos ES6 precisar ser servido por HTTP. Também aprendemos, na marra, sobre cache de navegador.

Conseguiríamos fazer um projeto parecido **sem** agentes? Honestamente, não nesse prazo nem com esse acabamento, dado o nosso ponto de partida (zero em jogos). Seria possível com muito estudo e tempo, mas o agente reduziu bastante a curva de aprendizado e o esforço de implementação.

4.3. Legibilidade e manutenção

Olhando o código final, a impressão é de que o resultado é **razoavelmente legível e organizado**: os arquivos têm responsabilidades separadas, os nomes costumam ser claros e os valores ajustáveis (custos, HP, velocidades, ondas) ficam reunidos em um arquivo de configuração, o que facilita mexer no balanceamento.

Ainda assim, achamos que **alguma refatoração seria saudável** se o projeto crescesse — por exemplo, o `main.js` concentra muita responsabilidade (loop, estados de tela, renderização) e poderia ser dividido. Para o escopo de uma atividade, porém, o nível de organização nos pareceu adequado.

4.4. Déficit de compreensão

Este é o ponto mais honesto a registrar. **Entendemos o projeto em nível geral** — sabemos o que cada arquivo faz, conseguimos localizar onde mexer para mudar um valor, um som ou um texto, e acompanhamos a lógica de alto nível. Mas **não entendemos em profundidade** as partes mais algorítmicas, como a geração procedural do labirinto e a busca de caminhos (BFS).

Para **modificações pequenas** (trocar sons, valores, textos, cores, ajustar balanceamento, adicionar um botão), provavelmente conseguiríamos mexer sozinhos, com algum esforço. Já mudanças **estruturais** (trocar o algoritmo de geração do mapa, reescrever o sistema de caminhos, mudar a arquitetura de telas) seriam difíceis sem o apoio do agente.

Sobre **como julgamos se uma modificação deu certo**: na prática, julgamos pelo **comportamento na tela** — rodando o jogo e vendo se faz o esperado (o som toca, a tela aparece, o caminho fica mais reto, o botão responde). É um critério funcional, mas limitado: ele não garante que o código por trás esteja correto, eficiente ou bem escrito, apenas que "parece funcionar".

4.5. Comentários finais

No geral, desenvolver com um agente de codificação foi uma experiência **muito positiva e produtiva**. O fluxo de "conversar, pedir, ver rodar e ajustar" é rápido e motivador, e o agente se saiu bem tanto em construir quanto em diagnosticar problemas a partir de descrições informais. Os principais aprendizados que levamos são: (1) a importância de ainda estudar os fundamentos para não ficar totalmente dependente; (2) o cuidado com o déficit de compreensão; e (3) o fato de que "funciona na tela" não é o mesmo que "o código está bom" — validar de verdade exige entender o que foi feito.